Pattern Matching

Globbing is a shell command-parsing operation that expands a wildcard pattern into a list of matching path names. Before command execution, the shell replaces the command-line metacharacters by the match list. Patterns that do not return matches display the original pattern request as literal text. The following table lists common metacharacters and pattern classes.

Pattern	Matches
*	Any string of zero or more characters.
?	Any single character.

```
[student@node1 ~]$ touch a1 a12 a123 b1 b12 b123 doc1 doc2
[student@node1 ~]$
[student@node1 ~]$ ls
a1  a12  a123  b1  b12  b123  database  doc1  doc2
[student@node1 ~]$
[student@node1 ~]$ ls a*
a1  a12  a123
[student@node1 ~]$ ls a?
a1
[student@node1 ~]$ ls a??
a12
[student@node1 ~]$ ls a???
a123
```

[*abc...*]	Any one character in the enclosed class (between the square brackets).

```
[student@node1 ~]$ cd database
[student@node1 database]$
[student@node1 database]$ touch a1 b1 c1 d1
[student@node1 database]$ ls
a1  b1  c1  d1
[student@node1 database]$ ls [a]*
a1
[student@node1 database]$ touch a12 b12 c12 d12
[student@node1 database]$
[student@node1 database]$ ls [a]*
a1  a12
[student@node1 database]$ ls [ab]*
a1  a12  b1  b12
[student@node1 database]$
[student@node1 database]$ ls [a]?
a1
[student@node1 database]$ ls [a]??
a12
[student@node1 database]$
[student@node1 database]$ ls [ab]?
a1  b1
[student@node1 database]$ ls [ab]??
a12  b12
[student@node1 database]$
```

[!*abc*...]	Any one character *not* in the enclosed class.

```
[student@node1 database]$ ls
a1  a12  b1  b12  c1  c12  d1  d12
[student@node1 database]$
[student@node1 database]$ ls [!a]*
b1  b12  c1  c12  d1  d12
[student@node1 database]$
[student@node1 database]$ ls [!ab]*
c1  c12  d1  d12
[student@node1 database]$
[student@node1 database]$ ls [!abc]*
d1  d12
[student@node1 database]$
```

```
[student@node1 database]$ ls [!a]?
b1  c1  d1
[student@node1 database]$ ls [!ab]?
c1  d1
[student@node1 database]$ ls [!ab]??
c12  d12
[student@node1 database]$
[student@node1 database]$
```

[^abc…]	Any one character not in the enclosed class.

```
[student@node1 database]$ ls [^ab]*
c1  c12  d1  d12
[student@node1 database]$
[student@node1 database]$ ls [^ab]?
c1  d1
[student@node1 database]$ ls [^ab]??
c12  d12
[student@node1 database]$
```

[[:alpha:]]	Any alphabetic character

```
[student@node1 database]$ ls
12_2024  13_2024  14_2024  a1  a12  b1  b12  c1  c12  d1  d12
[student@node1 database]$
[student@node1 database]$ ls [[:alpha:]]*
a1  a12  b1  b12  c1  c12  d1  d12
[student@node1 database]$
[student@node1 database]$ ls [^[:alpha:]]*
12_2024  13_2024  14_2024
[student@node1 database]$
[student@node1 database]$ ls [![:alpha:]]*
12_2024  13_2024  14_2024
[student@node1 database]$
```

[[:upper:]]	Any uppercase character.

```
[student@node1 database]$ touch DOC1 DOC2
[student@node1 database]$
[student@node1 database]$ ls [[:upper:]]*
DOC1  DOC2
[student@node1 database]$
[student@node1 database]$ ls [^[:upper:]]*
12_2024  13_2024  14_2024  a1  a12  b1  b12  c1  c12  d1  d12
```

[[:lower:]]	Any lowercase character

```
[student@node1 database]$ ls [[:lower:]]*
a1  a12  b1  b12  c1  c12  d1  d12
[student@node1 database]$
[student@node1 database]$ ls [^[:lower:]]*
12_2024  13_2024  14_2024  DOC1  DOC2
[student@node1 database]$
```

[[:digit:]]	Any single digit from 0 to 9

```
[student@node1 database]$ ls [[:digit:]]*
12_2024  13_2024  14_2024
[student@node1 database]$
[student@node1 database]$
```

[[:punct:]]	Any printable character that is not a space or alphanumeric

```
[student@node1 database]$ ls [[:punct:]]*
,mp3
[student@node1 database]$
```

Brace Expansion

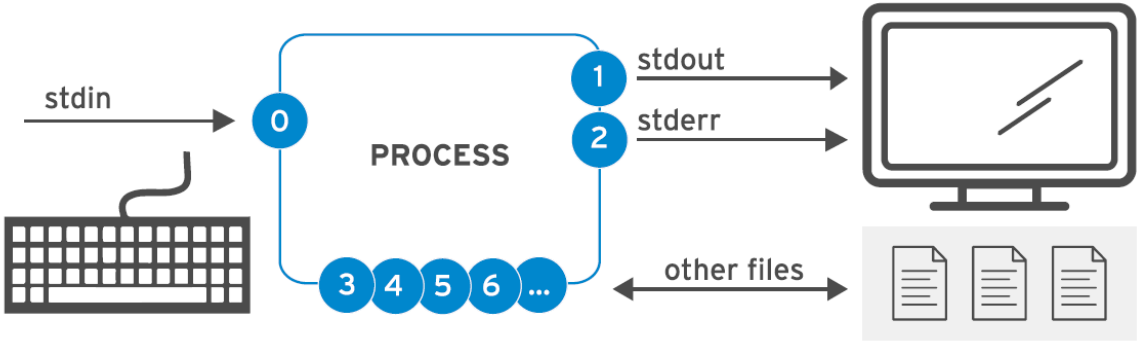
Brace expansion is used to generate discretionary strings of characters. Braces contain a comma separated list of strings, or a sequence expression. The result includes the text that precedes or follows the brace definition. Brace expansions might be nested, one inside another. You can also use double-dot syntax (..), which expands to a sequence. For example, the {m..p} double-dot syntax inside braces expands to m n o p.

```
[student@node1 brace]$ touch {a,c}.mp3
[student@node1 brace]$ ls
a.mp3  c.mp3
[student@node1 brace]$
[student@node1 brace]$ touch {g..j}.mp4
[student@node1 brace]$ ls
a.mp3  c.mp3  g.mp4  h.mp4  i.mp4  j.mp4
[student@node1 brace]$
```

```
[student@node1 brace]$ touch {a,c}.mp3
[student@node1 brace]$ ls
a.mp3  c.mp3
[student@node1 brace]$
[student@node1 brace]$ touch {g..j}.mp4
[student@node1 brace]$ ls
a.mp3  c.mp3  g.mp4  h.mp4  i.mp4  j.mp4
[student@node1 brace]$
[student@node1 brace]$ touch file{1..3}{a..c}.mp3
[student@node1 brace]$
[student@node1 brace]$ ls
a.mp3  file1a.mp3  file1c.mp3  file2b.mp3  file3a.mp3  file3c.mp3  h.mp4  j.mp4
c.mp3  file1b.mp3  file2a.mp3  file2c.mp3  file3b.mp3  g.mp4       i.mp4
[student@node1 brace]$
[student@node1 brace]$ touch doc{1,2}{a,b}
[student@node1 brace]$
[student@node1 brace]$ ls doc*
doc1a  doc1b  doc2a  doc2b
[student@node1 brace]$
[student@node1 brace]$ touch doc{3,4}{c,d}.mp3
[student@node1 brace]$ ls doc*
doc1a  doc1b  doc2a  doc2b  doc3c.mp3  doc3d.mp3  doc4c.mp3  doc4d.mp3
[student@node1 brace]$
[student@node1 brace]$ touch doc{5,6}{e,f}.{mp3,mp4}
[student@node1 brace]$ ls doc*
doc1a  doc2a  doc3c.mp3  doc4c.mp3  doc5e.mp3  doc5f.mp3  doc6e.mp3  doc6f.mp3
doc1b  doc2b  doc3d.mp3  doc4d.mp3  doc5e.mp4  doc5f.mp4  doc6e.mp4  doc6f.mp4
[student@node1 brace]$
```

Redirection

- A process uses numbered channels called ***file descriptors*** to get input and send output.
- (channel **0**) reads input from the keyboard. ***Standard input*** **[STDIN]**
- (channel **1**) sends normal output to the terminal. ***Standard output*** **[STDOUT]** (channel **2**) sends error messages to the terminal **[STDERR]**

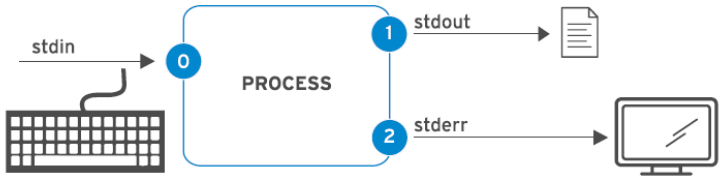
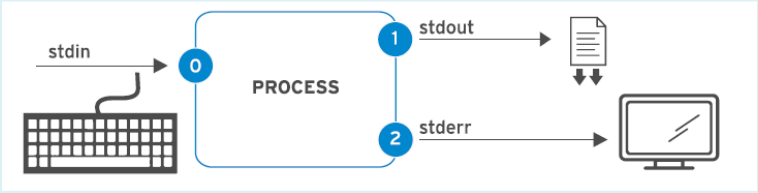
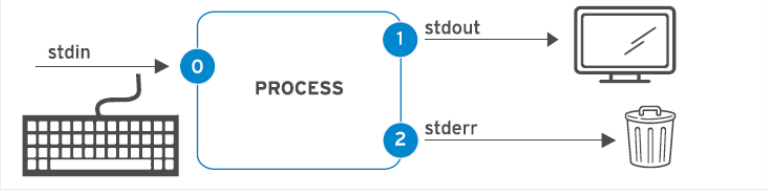
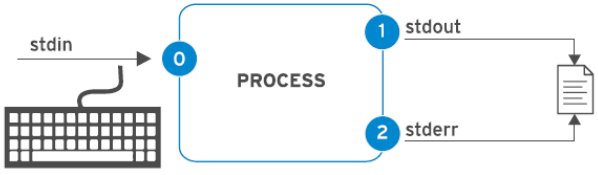


Process I/O channels (file descriptors)

Redirecting Output to a file

- I/O redirection changes how the process gets its input or output. Instead of getting input from the keyboard, or sending output and errors to the terminal, the process reads from or writes to file.
- Redirecting stdout suppresses process output from appearing on the terminal.
- If you want to discard messages, the special file ***/dev/null*** quietly discards channel output redirected to it and is always an empty file.

Output Redirection Operators

USAGE	EXPLANATION	VISUAL AID
> file	redirect **`stdout`** to overwrite a file	
>> file	redirect **`stdout`** to append to a file	
2> /dev/null	discard **`stderr`** error messages by redirecting to **`/dev/null`**	
> file ***2>&1***	redirect **`stdout`** and **`stderr`** to overwrite the same file	
&> file		

```
[student@node1 ~]$ find /etc/ -name passwd 1> output.txt
[student@node1 ~]$ find /etc/ -name passwd 1> output.txt 2>error1.txt
[student@node1 ~]$
[student@node1 ~]$
[student@node1 ~]$ find /etc/ -name fstab 1> rohit.output 2> /dev/null
[student@node1 ~]$
[student@node1 ~]$ cat rohit.output
/etc/fstab
[student@node1 ~]$
[student@node1 ~]$ cat /dev/null
[student@node1 ~]$
```